

Implementación de la Criba de Eratóstenes Distribuida mediante el Estándar MPI

Iver Samuel Medina Balboa¹, Andrés Maximiliano Espinoza Romero²,
Aron Donovan Costas Medrano³, Maximiliano Gómez Mallo⁴

Carrera de Informática, Universidad Mayor de San Andrés
¹6721489 ²6783133 ³9939515 ⁴14480221

Abstract—En este trabajo se presenta la implementación paralela de la Criba de Eratóstenes utilizando el estándar *Message Passing Interface* (MPI). El algoritmo divide el rango de búsqueda $[2, N]$ en segmentos equitativos distribuidos entre los procesos disponibles; los primos base hasta $\sqrt{N}$ se envían a cada proceso mediante comunicación punto a punto (`MPI_Send / MPI_Recv`) y cada proceso tamiza su segmento de forma independiente. Se evalúa el rendimiento con $N = 10,000,000$ para 1, 2 y 4 procesos, midiendo el tiempo de ejecución con `MPI_Wtime`. Los resultados muestran un speedup cercano al ideal con 2 procesos (1.997) que se vuelve sublineal con 4 (2.474), comportamiento esperable dado el reducido tamaño del problema, donde la fracción secuencial y el costo de comunicación punto a punto pesan más a medida que crece el número de procesos.

Index Terms—Criba de Eratóstenes, MPI, computación paralela, números primos, speedup, criba segmentada, criptografía, primos de Mersenne.

I. INTRODUCCIÓN

La obtención de números primos es un problema clásico en matemáticas computacionales con aplicaciones en criptografía, teoría de números y algoritmos de factorización [1]. El algoritmo más conocido para este fin es la *Criba de Eratóstenes*, propuesto alrededor del año 200 a.C. por el matemático griego del mismo nombre, y cuya complejidad asintótica es $O(N \log \log N)$.

Para valores grandes de N (del orden de 10^7 a 10^{10}), la versión secuencial de la criba se vuelve costosa tanto en tiempo como en memoria. La computación paralela ofrece una vía natural para escalar el algoritmo: dividir el rango de búsqueda entre múltiples procesos reduce la memoria por proceso y distribuye el trabajo de marcado de compuestos.

El estándar MPI (*Message Passing Interface*) [5] es la herramienta dominante en sistemas de memoria distribuida. A diferencia de OpenMP, donde los hilos comparten la misma memoria, los procesos MPI se ejecutan en espacios de memoria separados y se comunican mediante paso de mensajes. Este modelo es adecuado tanto para nodos individuales con múltiples núcleos como para clústeres de computadoras.

El objetivo de este trabajo es:

- 1) Implementar la criba segmentada distribuida con MPI en un único archivo `.c`, priorizando legibilidad.
- 2) Medir el speedup obtenido al escalar de 1 a 4 procesos.
- 3) Generar resultados reproducibles y visualizarlos con Gnuplot.

II. METODOLOGÍA

A. Descripción del algoritmo

La criba de Eratóstenes secuencial opera sobre un arreglo booleano de N elementos inicializados en “primo” (falso). Para cada entero p desde 2 hasta $\sqrt{N}$, si p sigue marcado como primo, se marcan como compuestos todos sus múltiplos a partir de p^2 .

La variante *segmentada* divide el dominio $[2, N]$ en bloques y procesa un bloque a la vez, reduciendo la presión sobre la caché. Esta misma idea se extiende al modelo paralelo: cada proceso MPI actúa sobre un bloque (segmento) del rango total.

B. Estrategia de distribución del rango

El rango $[2, N]$ se divide en `size` partes de tamaño aproximadamente igual, donde `size` es el número de procesos MPI. El proceso con identificador `rank` maneja el segmento:

$$\text{low}_i = 2 + \left\lfloor \frac{i \cdot (N - 1)}{\text{size}} \right\rfloor \quad (1)$$

$$\text{high}_i = \begin{cases} N & \text{si } i = \text{size} - 1 \\ \text{low}_{i+1} - 1 & \text{en otro caso} \end{cases} \quad (2)$$

Esta distribución garantiza que no existan huecos ni solapamientos entre segmentos, y que la carga sea balanceada (la densidad de primos decrece logarítmicamente, lo que produce segmentos casi uniformes para N moderados).

C. Comunicación entre procesos

El flujo de comunicación MPI del programa se resume en la Tabla I.

A diferencia de las operaciones colectivas, en este trabajo toda la comunicación se realiza *punto a punto*. La operación clave es el reparto de los primos base: `rank 0` calcula los primos hasta $\sqrt{N}$ (en nuestro caso, 446 primos para

TABLE I
OPERACIONES MPI UTILIZADAS

Operación	Dirección	Propósito
MPI_Init	local	Arrancar el entorno MPI
MPI_Comm_rank	local	Identificador del proceso
MPI_Comm_size	local	Total de procesos
MPI_Send	0 → resto	Enviar primos base
MPI_Recv	resto → 0	Recibir primos / conteos
MPI_Wtime	local	Medir tiempo de ejecución
MPI_Finalize	local	Cerrar el entorno MPI

$N = 10,000,000$) y los envía a cada proceso con un bucle de llamadas MPI_Send; cada proceso receptor los obtiene con MPI_Recv. Sin estos primos base, ningún proceso podría saber qué múltiplos marcar en su segmento [2], [3]. De forma simétrica, al final cada proceso envía su conteo local con MPI_Send y rank 0 los recibe uno a uno con MPI_Recv. Se optó por comunicación punto a punto en lugar de las primitivas colectivas MPI_Bcast y MPI_Gather para mostrar explícitamente el patrón emisor-receptor que subyace a toda la comunicación en MPI.

D. Pseudocódigo

El Algoritmo 1 describe la estructura del programa en pseudocódigo, independiente de cualquier lenguaje de implementación. Las operaciones de comunicación punto a punto de MPI aparecen como pasos descriptivos; su semántica se detalla en la Sección II-E.

E. Funciones MPI utilizadas

El programa se apoya en un subconjunto reducido de funciones del estándar MPI [2], [5]. Todas operan sobre el comunicador global MPI_COMM_WORLD, que agrupa a la totalidad de los procesos lanzados. A continuación se describe el propósito de cada una:

- MPI_Init / MPI_Finalize: arrancan y cierran el entorno de comunicación. La primera debe ser la primera llamada MPI del programa y la segunda la última; todo proceso que inicializa MPI está obligado a finalizarlo para liberar los recursos internos.
- MPI_Comm_rank: devuelve el *rank*, el identificador único del proceso dentro del comunicador $(0, 1, \dots, size - 1)$. Es lo que permite que un mismo binario se comporte distinto en cada proceso.
- MPI_Comm_size: devuelve *size*, la cantidad total de procesos. Con él cada proceso calcula el tamaño de su segmento.
- MPI_Send: envío *bloqueante punto a punto*. Transmite un mensaje desde el proceso que la invoca hacia un único destino, identificado por su *rank*. Sus argumentos son el buffer de datos, la cantidad de elementos, el tipo, el rank destino, un *tag* (etiqueta entera que distingue mensajes) y el comunicador. Aquí rank 0 la usa dentro de un bucle para enviar los primos base a cada proceso; sin ellos, los procesos 1, 2, ... no sabrían qué múltiplos marcar. Recorrer los destinos con MPI_Send *emula* la difusión

Algoritmo 1 Criba de Eratóstenes distribuida con MPI

Entrada: Límite superior N

Salida: Cantidad total de primos en $[2, N]$

```

1: Inicializar el entorno MPI {MPI_Init}
2: rank ← identificador del proceso {MPI_Comm_rank}
3: size ← número total de procesos {MPI_Comm_size}
4:  $t_{ini}$  ← tiempo actual {MPI_Wtime}
5:
6: si rank = 0 entonces
7:   primos_base ← criba secuencial sobre  $[2, \sqrt{N}]$ 
8:   para dest = 1 hasta size - 1 hacer
9:     enviar base_count y primos_base a dest
       {MPI_Send}
10:  fin para
11: si no
12:  recibir base_count y primos_base de rank 0
    {MPI_Recv}
13: fin si
14:
15: low ←  $2 + rank \cdot (N - 1) / size$ 
16: high ← fin del segmento de este proceso
17: para cada primo  $p \in primos\_base$  hacer
18:   marcar como compuestos los múltiplos de  $p$  en
     [low, high]
19: fin para
20: conteo ← número de no marcados en el segmento
21:
22: si rank = 0 entonces
23:  recibir cada conteo de los procesos  $1 \dots size - 1$ 
    {MPI_Recv}
24: si no
25:  enviar conteo local a rank 0 {MPI_Send}
26: fin si
27:  $t_{fin}$  ← tiempo actual {MPI_Wtime}
28: si rank = 0 entonces
29:  imprimir resultados y escribir resultados.dat
30: fin si
31: Cerrar el entorno MPI {MPI_Finalize}

```

que haría una primitiva colectiva, pero deja explícito cada par emisor-receptor.

- MPI_Recv: recepción *bloqueante punto a punto*, la contraparte de MPI_Send. Espera un mensaje de un origen y *tag* dados y lo deposita en un buffer; el proceso queda detenido hasta que el mensaje llega. Cada par Send/Recv debe coincidir en tipo, tamaño, *tag* y comunicador. Los procesos la usan para recibir los primos base de rank 0; y al final, rank 0 la invoca en un bucle para recolectar el conteo local de cada proceso, ordenándolos por *rank* para luego sumarlos y obtener el total.
- MPI_Wtime: devuelve un instante de tiempo de alta resolución en segundos; la diferencia entre dos llamadas mide el tiempo de ejecución empleado para calcular el *speedup*.

F. Marcado de múltiplos en el segmento

Para un primo base p y un segmento $[low, high]$, el primer múltiplo de p que pertenece al segmento es:

$$start = \left\lceil \frac{low}{p} \right\rceil \cdot p \quad (3)$$

Si $start == p$ (es decir, p cae dentro del segmento), se avanza un paso más ($start += p$) para no marcar al primo mismo como compuesto.

III. RESULTADOS Y DISCUSIÓN

A. Correctitud

El número de primos menores o iguales a 10,000,000 es 664,579 (valor exacto conocido como $\pi(10^7)$ [6]). La implementación produce este valor exacto independientemente del número de procesos, lo que valida la correctitud de la distribución y del algoritmo de marcado.

B. Rendimiento

La Tabla II muestra los tiempos de ejecución y el speedup obtenidos al ejecutar el programa con 1, 2 y 4 procesos MPI.

TABLE II
TIEMPOS DE EJECUCIÓN Y SPEEDUP PARA $N = 10,000,000$

Procesos	Tiempo (s)	Speedup
1	0.0234	1.000
2	0.0117	1.997
4	0.0094	2.474

C. Distribución de primos por proceso

La Fig. 1 muestra la cantidad de primos encontrados por cada proceso cuando se utilizan 4 procesos. La distribución es relativamente uniforme, lo que confirma que la estrategia de partición por rango produce una carga balanceada: la densidad de primos disminuye gradualmente con N (ley del número primo [1]), pero la diferencia entre segmentos es pequeña para $N = 10^7$.

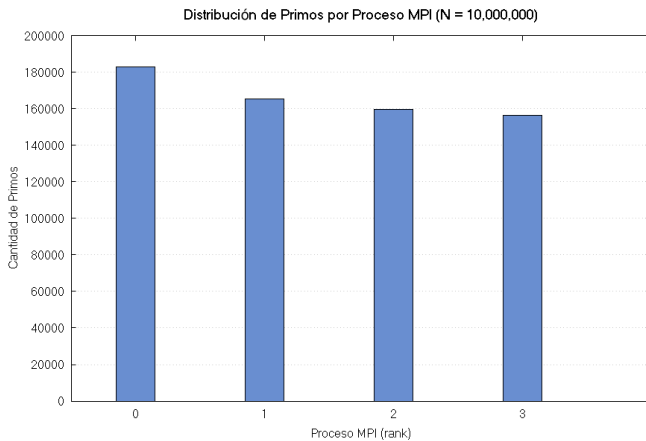


Fig. 1. Primos encontrados por cada proceso MPI (4 procesos, $N = 10^7$).

D. Curva de speedup

La Fig. 2 compara el speedup real obtenido con el speedup ideal $S(p) = p$. El speedup real es sublineal debido a:

- La fracción secuencial del programa (cálculo de primos base en rank 0 y escritura de archivo), regida por la ley de Amdahl.
- La latencia de las llamadas punto a punto MPI_Send y MPI_Recv: rank 0 emite y recibe $O(p)$ mensajes de forma secuencial, por lo que el costo de comunicación crece linealmente con el número de procesos p .
- El overhead del sistema operativo al lanzar procesos MPI en una sola máquina compartiendo memoria física.

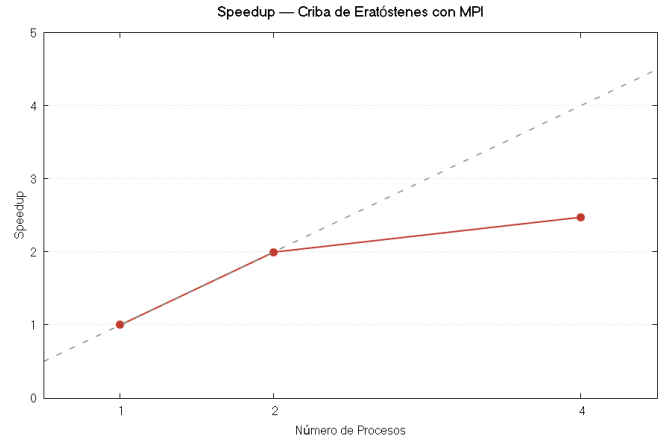


Fig. 2. Speedup real vs. ideal para la criba distribuida.

IV. APLICACIONES EN EL MUNDO REAL

Aunque el problema de generar números primos pueda parecer puramente académico, tanto el resultado del algoritmo como la técnica de paralelización empleada tienen aplicaciones directas en la industria y la ciencia.

A. Aplicaciones de los números primos y la criba

- **Criptografía de clave pública (RSA).** La seguridad del criptosistema RSA se basa en la dificultad de factorizar el producto de dos primos grandes: multiplicar dos primos de cientos de dígitos es trivial, pero recuperar los factores a partir del producto es computacionalmente inviable [1]. Generar esas claves exige producir y verificar números primos muy grandes, tarea para la que las cribas (combinadas con tests de primalidad) son un primer filtro eficiente.
- **Firmas digitales y certificados.** Los esquemas de firma (RSA, DSA) y la infraestructura de certificados que sustenta HTTPS y la banca en línea descansan sobre la misma aritmética modular de números primos.
- **Funciones y tablas hash.** Es habitual usar tamaños de tabla y módulos primos para dispersar las claves de manera uniforme y reducir colisiones, lo que mejora el rendimiento de estructuras de datos fundamentales.

- **Generadores de números pseudoaleatorios.** Muchos generadores (p.ej. congruenciales lineales) emplean módulos primos para maximizar el período de la secuencia generada.
- **Códigos de detección y corrección de errores** y otras áreas de la teoría de números computacional emplean aritmética sobre cuerpos finitos de orden primo.

B. Cómputo intensivo de primos: proyectos de cálculo distribuido

El mismo patrón empleado en este trabajo —repartir el rango entre procesos (*scatter*), cribar localmente (*compute*) y recolectar (*gather*)— es la base de varios proyectos reales de investigación en teoría de números, donde el volumen de cómputo obliga a paralelizar y a apoyarse en cribas:

- **GIMPS (Great Internet Mersenne Prime Search).** Búsqueda distribuida de primos de Mersenne, de la forma $2^p - 1$. Los mayores primos conocidos pertenecen a esta familia: el récord actual supera los 41 millones de dígitos [8]. Miles de computadoras voluntarias se reparten los exponentes candidatos. Las cribas intervienen al generar y pre-filtrar dichos exponentes (que deben ser primos) y en el *trial factoring*; la confirmación final de primalidad se realiza con el test de Lucas–Lehmer.
- **PrimeGrid.** Proyecto de cómputo distribuido cuyos subproyectos de *sieving* ejecutan, literalmente, cribas para descartar candidatos con factores pequeños antes de los tests de primalidad. Es el análogo más directo a la criba segmentada de este trabajo, repartida entre nodos de todo el mundo para hallar primos gemelos, de Sophie Germain, primoriales y factoriales.
- **Verificación de conjeturas.** La conjetura de Goldbach se ha verificado computacionalmente hasta 4×10^{18} [9] y la de los primos gemelos hasta cotas igualmente enormes. Ello exige *generar todos los primos* hasta esos límites mediante cribas segmentadas paralelas, exactamente la técnica aquí implementada.
- **Cálculo de $\pi(x)$ y brechas entre primos.** Determinar la función de conteo de primos o localizar brechas récord (*prime gaps*) obliga a tamizar rangos colosales, una tarea naturalmente paralelizable por segmentos.
- **Factorización mediante cribas.** Los algoritmos de factorización más potentes —la criba del cuerpo de números (GNFS) y la criba cuadrática— son, en esencia, cribas masivamente paralelas. Con ellas se factorizó RSA-250 en 2020. Esto cierra el círculo con la criptografía: romper RSA es, en la práctica, otra gran criba distribuida.

En todos estos casos, la criba segmentada distribuida de este trabajo —con un resultado verificable, $\pi(10^7) = 664,579$ — constituye una versión a pequeña escala del mismo patrón que la investigación matemática lleva hasta límites de 10^{18} y más allá.

V. CONCLUSIONES

Se implementó exitosamente la Criba de Eratóstenes distribuida utilizando MPI, obteniendo el resultado correcto

de $\pi(10^7) = 664,579$ primos en todos los experimentos.

El speedup alcanzado es cercano al ideal con 2 procesos (1.997) y se vuelve sublineal con 4 (2.474). Esto indica que, para $N = 10^7$, la fracción paralelizable es dominante pero no abrumadora: las principales limitaciones provienen de la porción secuencial (criba base) y del overhead de comunicación punto a punto, que crece con el número de procesos, ambas moduladas por la ley de Amdahl.

La implementación en un único archivo .c con comentarios extensos demuestra que los patrones fundamentales de MPI —inicialización, envío y recepción punto a punto (MPI_Send / MPI_Recv)— pueden expresarse de forma clara y legible sin sacrificar correctitud, y que el reparto y la recolección de datos pueden construirse con primitivas punto a punto sin recurrir a operaciones colectivas.

Como trabajo futuro se podría explorar:

- Aumentar N a 10^9 o 10^{10} para revelar la escalabilidad en rangos donde la fracción secuencial se vuelve negligible.
- Combinar MPI con OpenMP (modelo híbrido) para explotar múltiples núcleos dentro de cada nodo.
- Aplicar técnicas de rueda de factorización (*wheel factorization*) [7] para reducir el trabajo de marcado.

REFERENCES

- [1] D. E. Knuth, *The Art of Computer Programming*, Vol. 2: *Seminumerical Algorithms*, 3rd ed. Reading, MA: Addison-Wesley, 1997.
- [2] W. Gropp, E. Lusk, and A. Skjellum, *Using MPI: Portable Parallel Programming with the Message Passing Interface*, 2nd ed. Cambridge, MA: MIT Press, 1999.
- [3] M. J. Quinn, *Parallel Programming in C with MPI and OpenMP*. New York, NY: McGraw-Hill, 2003.
- [4] P. S. Pacheco, *An Introduction to Parallel Programming*. Burlington, MA: Morgan Kaufmann, 2011.
- [5] Message Passing Interface Forum, “MPI: A Message-Passing Interface Standard, Version 4.0,” Jun. 2021. [Online]. Available: <https://www.mpi-forum.org/docs/mpi-4.0/>
- [6] E. Bach and J. Shallit, *Algorithmic Number Theory, Vol. 1: Efficient Algorithms*. Cambridge, MA: MIT Press, 1996.
- [7] P. Pritchard, “Explaining the wheel factorization,” *BIT Numerical Mathematics*, vol. 22, no. 4, pp. 442–454, 1982.
- [8] Great Internet Mersenne Prime Search (GIMPS), “Distributed computing project to search for Mersenne prime numbers.” [Online]. Available: <https://www.mersenne.org/>
- [9] T. Oliveira e Silva, S. Herzog, and S. Pardi, “Empirical verification of the even Goldbach conjecture and computation of prime gaps up to $4 \cdot 10^{18}$,” *Mathematics of Computation*, vol. 83, no. 288, pp. 2033–2060, 2014.